

HW 5

Sam Ly

March 6, 2026

Chapter 10

<https://github.com/samlyme/Assignments/tree/cs4080/ch10/challenge/CS4080>

1. Our interpreter carefully checks that the number of arguments passed to a function matches the number of parameters it expects. Since this check is done at runtime on every call, it has a performance cost. Smalltalk implementations don't have that problem. Why not?

Smalltalk doesn't have this problem because you don't actually specify a "name" for methods in Smalltalk. Function calling is implemented as a message passing system, so the shape of the message itself determines what code is called.

2. Lox's function declaration syntax performs two independent operations. It creates a function and also binds it to a name. This improves usability for the common case where you do want to associate a name with the function. But in functional-styled code, you often want to create a function to immediately pass it to some other function or return it. In that case, it doesn't need a name.

Languages that encourage a functional style usually support anonymous functions or lambdas—an expression syntax that creates a function without binding it to a name. Add anonymous function syntax to Lox so that this works:

```
fun thrice(fn) {
  for (var i = 1; i <= 3; i = i + 1) {
    fn(i);
  }
}

thrice(fun (a) {
  print a;
});
// "1".
// "2".
// "3".
```

First, we create a new Expr type in the GenerateAST class called 'Lambda'. Then, we create a new class 'LoxLambda' which is similar to the existing LoxFunction class, just without the 'name' field. After that, in the interpreter, visit the lambda expression by returning an instance of 'LoxLambda'.

```
@Override
public Object visitLambdaExpr(Expr.Lambda expr) {
    return new LoxLambda(expr.params, expr.body, environment);
}
```

The trickiest part of this challenge was the parser, because of the weird expression-statement case for anonymous functions. As an aside, this syntax literally does nothing, even if the params and body are defined, the lambda returned is instantly discarded. But the way I approached this was to create a new expression parser function 'lambda' that is basically the same as the existing 'function' statement parser.

```
private Expr expression() {
    return lambda();
}

private Expr lambda() {
    if (!match(FUN)) return assignment();

    consume(LEFT_PAREN, "Expect '(' after lambda start.");
    List<Token> parameters = new ArrayList<>();
    if (!check(RIGHT_PAREN)) {
        do {
            if (parameters.size() >= 255) {
                //noinspection ThrowableNotThrown
                error(peek(), "Can't have more than 255 parameters.");
            }

            parameters.add(
                consume(IDENTIFIER, "Expect parameter name."));
        } while (match(COMMA));
    }
    consume(RIGHT_PAREN, "Expect ')' after parameters.");

    consume(LEFT_BRACE, "Expect '{' before lambda body.");
    List<Stmt> body = block();
    return new Expr.Lambda(parameters, body);
}
```

Then, when parsing for declaration, we do this check for the weird edge case

```
private Stmt declaration() {
    try {
        if (match(FUN)) {
            if (check(IDENTIFIER)) return function("function");
            current--;
        }
        if (match(VAR)) return varDeclaration();
        return statement();
    } catch (ParseError error) {
        synchronize();
        return null;
    }
}
```

If the name isn't there, then the execution falls through and treats the 'fun' keyword as the start of a lambda expression.

3. Is this program valid?

```

fun scope(a) {
  var a = "local";
}

```

In other words, are a function's parameters in the same scope as its local variables, or in an outer scope? What does Lox do? What about other languages you are familiar with? What do you think a language should do?

This is perfectly valid because our implementation treats the parameter names almost like an inherited scope variable. With our block scoping, we can shadow already-defined variables, so this is no different. In my opinion, the way Lox handles this is how languages should handle this anyway. Maybe this is bad code style, but that is easily solved by linters in production languages.

Chapter 11

1. Why is it safe to eagerly define the variable bound to a function's name when other variables must wait until after they are initialized before they can be used?

As opposed to variable declarations, the function body isn't evaluated until it is called. Thus, nothing can change between declaration and definition.

2. How do other languages you know handle local variables that refer to the same name in their initializer, like:

```

var a = "outer";
{
  var a = a;
}

```

Is it a runtime error? Compile error? Allowed? Do they treat global variables differently? Do you agree with their choices? Justify your answer.

Other languages typically just let it happen. They define the inner scope `a` to have the same initial value as the outer scope `a`. Then, if the inner scope mutates `a` with `a = "new value"`, the other scope is unaffected. I feel like this makes more sense, as it lets us clearly define boundaries, even when the names are the same.

3. Extend the resolver to report an error if a local variable is never used.

Done. The code can be found in the github.

4. Our resolver calculates which environment the variable is found in, but it's still looked up by name in that map. A more efficient environment representation would store local variables in an array and look them up by index.

Extend the resolver to associate a unique index for each local variable declared in a scope. When resolving a variable access, look up both the scope the variable is in and its index and store that. In the interpreter, use that to quickly access a variable by its index instead of using a map.

Done. The code can be found in the github.